

Cours sur le langage C (Arduino)

Le langage C

Le langage C a été inventé au cours de l'année 1972 dans les Laboratoires Bell. Il était développé en même temps que UNIX par Dennis Ritchie et Ken Thompson. Ken Thompson avait développé un prédécesseur de C, le langage B, qui est lui-même inspiré de BCPL. Dennis Ritchie a fait évoluer le langage B dans une nouvelle version suffisamment différente, en ajoutant notamment les types, pour qu'elle soit appelée C¹.

Par la suite, Brian Kernighan aida à populariser le langage C. Il procéda aussi à quelques modifications de dernière minute.

En 1978, Kernighan fut le principal auteur du livre *The C Programming Language* décrivant le langage enfin stabilisé ; Ritchie s'était occupé des appendices et des exemples avec Unix. On appelle aussi ce livre « le K&R », et l'on parle de C traditionnel ou de C K&R lorsqu'on se réfère au langage tel qu'il existait à cette époque

Qualités et défauts

C'est un des langages les plus utilisés car :

- il existe depuis le début des années 1970 ;
- il est fondé sur un standard ouvert ;
- de nombreux informaticiens le connaissent ;
- étant du plus bas niveau portable possible, il permet la minimisation de l'allocation mémoire nécessaire et la maximisation de la performance, notamment par l'utilisation de pointeurs ;
- des compilateurs et bibliothèques logicielles existent sur la plupart des architectures ;
- il a influencé de nombreux langages plus récents dont C++, Java et PHP ; sa syntaxe en particulier est largement reprise ;
- il met en œuvre un nombre restreint de concepts, ce qui facilite sa maîtrise et l'écriture de compilateurs simples et rapides ;
- il ne spécifie pas rigidelement le comportement du fichier exécutable produit, ce qui aide à tirer parti des capacités propres à chaque ordinateur ;
- il permet l'écriture de logiciels qui n'ont besoin d'aucun support à l'exécution (ni bibliothèque logicielle ni machine virtuelle), au comportement prévisible en temps d'exécution comme en consommation de mémoire vive, comme des noyaux de système d'exploitation et des logiciels embarqués.

Ses principaux inconvénients sont :

- le peu de vérifications offertes à par les compilateurs d'origine (K&R C), et l'absence de vérifications à l'exécution, ce qui fait que des erreurs qui pourraient être automatiquement détectées lors du développement ne le sont que plus tard, souvent au prix d'un plantage du logiciel ;
 - des vérifications sont ajoutées avec le temps, mais elles restent partielles ;
- son approche de la modularité restée au niveau de ce qui se faisait au début des années 1970, et largement dépassée depuis par d'autres langages :
 - il ne facilite pas la programmation orientée objet ;
 - il ne permet pas de créer des espaces de noms ;
- la gestion d'exceptions très sommaire ;
- le support très limité de la généricité, malgré l'introduction des expressions à type générique en C11 ;
- les subtilités de l'écriture de programmes portables, car le comportement exact des exécutables dépend de l'ordinateur cible ;
- le support minimaliste de l'allocation de mémoire et des chaîne de caractères, ce qui oblige les programmeurs à s'occuper de détails fastidieux et sources de bugs ; il n'y a notamment pas de ramasse-miettes standard (ramasse miettes → système de gestion de la mémoire allouée puis inutilisées) ;
- les bugs graves qui peuvent être causés par un simple manque d'attention du développeur ; tel le dépassement de tampon qui constitue une faille de sécurité informatique exploitable par les logiciels malveillants ;
- certaines erreurs ne peuvent être détectées automatiquement qu'à l'aide d'outils supplémentaires et non standardisés, comme lint et Valgrind.

Utilisation en 1^{ère} STI2D

Plusieurs logiciels sont exploitables j'ai choisi CodeBlocs pour les raisons suivantes :

- **Il est gratuit**
- **Il est constamment remis à jour**
- **Il permet de programmer en C, C++**

Nous pourrions aussi utiliser Proteus

- Il intègre une interface de programmation en C pour les cibles PIC et AVR (Arduino)
- Il permet de simuler certains montages en temps réel
- Il est payant

1. Mots-clés

Le langage C est un langage par mots clés, ce qui signifie qu'un certain nombre de mots sont réservés pour le langage lui-même et ne peuvent donc pas être utilisés comme indicateurs.

La liste est la suivante :

auto	double	int	struct
break	else	long	switch
case	enum	register	typedef
char	extern	return	union
const	float	short	unsigned
continue	for	signed	void
default	goto	sizeof	volatile
do	if	static	while

2. Commentaires

Deux méthodes :

`/* commentaires */` → permet de commenter sur plusieurs lignes

`// commentaires` → commente sur une seule ligne

3. Les types de base

les types de base permettent de définir les variables déclarées en début de programme

Nom du type	Minimum	Maximum
<code>signed char</code>	-127	127
<code>int</code>	-32 767	32 767
<code>long</code>	-2 147 483 647	2 147 483 647
<code>float</code>	-1 x10 ³⁷	1 x10 ³⁷
<code>double</code>	-1 x10 ³⁷	1 x10 ³⁷

<code>unsigned char</code>	0 à 255
<code>unsigned int</code>	0 à 65 535
<code>unsigned long</code>	0 à 4 294 967 295

Type « *char* »

`char` → cet objet peut contenir le code de n'importe quel caractère (généralement ASCII) codé sur 8bits

exemple :

`char x,y ;` → signé
`char bonjour ;` → signé
`unsigned char z ;` → non-signé

(on peut écrire `x='g'` si x est déclaré en char, x contiendra donc l'équivalent de g en ASCII)

Type « *entier* »

`int` → cet objet contient un nombre sur 16 ou 32 bits selon le programme

Short int → cet objet contient un nombre sur 16

Long int → cet objet contient un nombre sur 32

(à vérifier long long int → 64bits)

(décimal → 235 nombre sans suffixe, hexadécimale → 0x25, binaire → 0b00011001 ou B00011001)

Type « flottants »

Type	Valeur minimale	Valeur maximale
float	1.175494351 E – 38	3.402823466 E + 38
double	2.2250738585072014 E – 308	1.7976931348623158 E + 308

Caractères particuliers utilisés souvent sous l'invite de commande

Constante	sémantique
'\n'	new line
'\t'	horizontal tabulation
'\v'	vertical tabulation
'\b'	back space
'\r'	carriage return
'\f'	form feed
'\a'	new line

Chaine de caractères

" Ceci est une chaine "

" Ceci est une chaine\n " → renvoie à la ligne

" Ceci est une chaine\t " → effectue une tabulation

Les #define

La syntaxe « #define » permet de définir le contenu d'une constante pour tout le programme

#define PI 3.1416 //pas de point-virgule

Opérations

$x=x+1;$ équivalent à $x+=1;$ équivalent à $x++;$

$x=x-1;$ équivalent à $x-=1;$ équivalent à $x--;$

$x=x*1;$

$x=x/1;$

Rappelons que la valeur de x (à gauche de l'expression) est remplacée par le contenu de l'opération (à droite de l'expression)

Donc si $x=5$ AVANT l'opération $x=x+1$ vaut $x=6$ APRES.

L'opérateur modulo (noté %) délivre le reste d'une division

Donc $x=12\%3$ délivre $x=0$

$x=12\%5$ délivre $x=2$

Comparaisons

opérateur	sémantique
>	Strictement supérieur
<	Strictement inférieur
>=	Supérieur ou égal
<=	Inférieur ou égal
==	égal
!=	différent

4. Les instructions les plus usuelles

Instruction if

```
if (a > b)
{
    if (c < d)
    {
        u = v;
    }
}
else
    i = j;
```

Instruction #include

Permet de déclarer une librairie utilisée dans tous le programme

```
#include <stdio.h> //appel de la librairie stdio permettant l'accès au clavier et la sortie à l'écran
```

Instruction for

Exemple :

```
y=0 ;
for(x=0 ; x<10 ; x++)
{
    y=y+1 ;
}
```

Dans cet exemple on effectue une boucle de 0 à 9 par pas de 1, le contenu de cette boucle effectuant l'incrément de y, à la fin y=10

Instruction while

```
y=0 ;  
while(x==1)           //noter le double =  
{  
    y++ ;  
    if(y==10)  
    {  
        x=1 ;  
    }  
}
```

Dans ce programme tant que x=1 la boucle s'effectue et y s'incrémente, dès que y=10 x=1 et le programme sort de la boucle

Le double égal (==) est un test

Le simple (=) est une affectation

Instruction do

```
Do (x++) while(x<10)
```

Ce programme incrémente x tant qu'il est inférieur à 10 strictement

Instruction break

Stop le processus do, while ou for avant la fin du processus

```
for (x=0 ; x<100 ; x++)  
{  
    if(x==50)  
    {  
        break ;  
    }  
}
```

Ce programme sort de la boucle for dès que x=50 et n'attend donc pas la fin du processus jusqu'à 100

Instruction ET et OU (opérateur logique vu en cours)

ET → &&

OU → || (alt gr 6)

NON → !

5. Les tableaux

Exemple :

```
int t[10];                /* tableau nommé t de 10 int */
long int t1[10], t2[20];  /* tableau notée t1 de 10 long int, tableau notée t2
de 20 long int */
```

Pour accéder à une cellule d'un tableau il suffit d'écrire $x=t[n]$ n étant le numéro de la cellule

En pratique, il est recommandé de toujours donner un nom à la constante qui indique le nombre d'éléments d'un tableau.

Exemples :

```
#define N 100
int t[N];
```

```
#define N 5
int t[N] = {1, 2, 3, 4, 5};    //permet de définir les valeurs d'un tableau
char tab[3] = {'a', 'b', 'c'}; //on peut aussi définir un tableau de caractères
```

mieux encore:

```
char tab[3] = « abc » ;      tab[0]='a'
                             tab[1]='b'
                             tab[2]='c'
```

```
char ch[] = "ch aura 22 caractères"; // ce tableau n'est pas quantifié mais le
compilateur comptabilisera // tout seul le nombre d'éléments + le caractère
null
```

6. Impression formatée

printf dépend de la librairie <stdio.h>

Format	Type attendu
"%d"	int
"%ld"	long
"%f"	float
"%f"	double

```
printf("bonjour"); //affiche bonjour sans aller à la ligne  
printf("bonjour\n"); //affiche bonjour en allant à la ligne
```

```
int i = 12;  
int j = 32;  
printf("la valeur de i est %d et celle de j est %d", i, j);
```

```
//affiche "la valeur de i est 12 et celle de j est 32" sans aller à la ligne
```

%d → decimal

%c → caractère

%x → hexadécimal

7. Structure d'un programme

Nous ne considérerons pour débiter que le cas de programmes formés d'une seule unité de compilation. Un programme C est une suite de déclarations de variables et de définitions de fonctions dans un ordre quelconque. Les variables sont des variables dont la durée de vie est égale à celle du programme, et qui sont accessibles par toutes les fonctions. Ces variables sont dites variables globales, par opposition aux variables déclarées à l'intérieur des fonctions qui sont dites locales.

Bien que ce ne soit pas obligatoire, il est généralement considéré comme étant un bon style de programmation de regrouper toutes les déclarations de variables en tête du programme.

La structure d'un programme devient alors la suivante :

```
int i,j;                /* i,j,a,b,c sont des variables globales */
double a,b,c;

void p1(int k)           /* début de la définition de la procédure p1 */
                        /* p1 a un seul paramètre entier : k */
{
    int s,t;             /* variables locales _a p1 */
    ...                 /* instructions de p1 qui peuvent accéder à */
    ...                 /* k,i,j,a,b,c,s et t */
}                       /* fin de la définition de p1 */

int f1(double x,double y) /* début de la définition de la fonction f1, */
                        /* f1 a deux paramètres flottants: x et y */
{
    double u,v;          /* variables locales _a f1 */
    ...                 /* instructions de f1 qui peuvent accéder à */
    ...                 /* x,y,i,j,a,b,c,u et v */
}                       /* fin de la définition de f1 */

int main()              /* début du main, il n'a pas de paramètre */
{
    ...                 /* instructions qui appellerons p1 et f1 */
    return(0);          /* doit se trouver en fin de programme pour
terminer*/
                        /* correctement*/
}                       /* fin de la définition de main */
```

PROCEDURE → ne renvoie rien → une procédure s'exécute, c'est tout!

FONCTION → renvoie un résultat en s'exécutant

Parmi les instructions d'une fonction, il doit y avoir au moins une instruction du type :

return expression ;

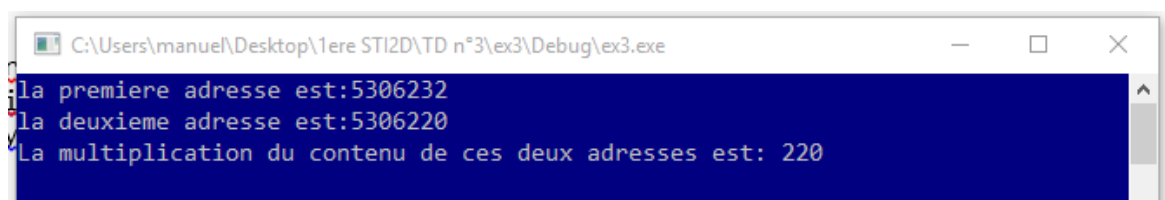
8. Notion de pointeur

Un pointeur est une adresse (et non une valeur) contenant une valeur

L'exemple suivant montre l'utilité d'un pointeur :

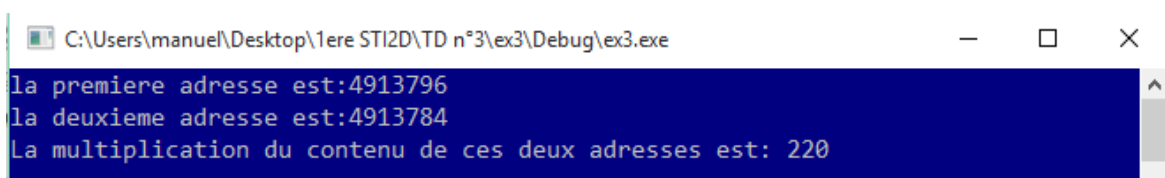
```
int main()
{
    int i,j;
    int *p,*z; //le symbole * désigne une adresse
    i = 10;
    j = 22;
    p = &i;
    z = &j;
    printf("la premiere adresse est:%d\n", p);
    printf("la deuxieme adresse est:%d\n", z);
    printf("La multiplication du contenu de ces deux adresses est:
%d\n", *p**z);
    while (1);
}
```

Le résultat est le suivant :



```
C:\Users\manuel\Desktop\1ere STI2D\TD n°3\ex3\Debug\ex3.exe
la premiere adresse est:5306232
la deuxieme adresse est:5306220
La multiplication du contenu de ces deux adresses est: 220
```

Si on relance le programme on obtient :



```
C:\Users\manuel\Desktop\1ere STI2D\TD n°3\ex3\Debug\ex3.exe
la premiere adresse est:4913796
la deuxieme adresse est:4913784
La multiplication du contenu de ces deux adresses est: 220
```

On s'aperçoit que c'est la machine qui affecte les adresses (elle cherche les adresses libres)

9. L'instruction switch

```
enum {BLEU=1, BLANC, ROUGE}; //Enumération de couleurs

void print_color(int color)      //procédure color
{
    switch(color)                //en fonction de la valeur color (0,1 ou 2) on
                                //affiche la bonne couleur
    {
        case BLEU :      printf("bleu");    break;
        case BLANC :     printf("blanc");    break;
        case ROUGE :     printf("rouge");    break;
        default : printf("erreur interne du logiciel numéro xx\n");
    }
}
```

10.Sortie scanf_s

scanf_s → permet de récupérer un caractère ou un chiffre du clavier
Syntaxe:

```
void main(void)
{
    int x;

    while (1)
    {
        scanf_s("%d", &x);
        if (x == 5)
        {
            printf("%d", 10);
        }
    }
}
```

Les tests, avec scanf_s, doivent être utilisés sous forme numérique (avec des déclarations de variables de type int comme dans l'exemple précédent) mais il est possible de tester un caractère unique

scanf_s entraîne automatiquement un passage à la ligne.

Exemple :

```
void main(void)
{
    char x,y;

    while (1)
    {
        scanf_s("%c", &x,1);
        if (x == 'm')
        {
            printf("%d", 10);
        }
    }
}
```

La syntaxe pour un mot est un peu plus compliquée :

```
scanf_s("%s", nom, sizeof(nom)) ;
```

nom est une chaîne de caractères

`sizeof(nom)` permet d'indiquer la taille du mot

Pour les accents :

<http://www.theasciicode.com.ar/extended-ascii-code/letter-e-acute-accent-e-acute-lowercase-ascii-code-130.html>

`printf(« b%cb%c », 130,130)` donnera « bébé »

Librairie `math.h`

fabs

Cette fonction retourne la valeur absolue d'un nombre, c'est-à-dire $|x|$ (c'est la notation mathématique).

La valeur absolue d'un nombre est sa valeur positive

ceil

Cette fonction renvoie le premier nombre entier après le nombre décimal qu'on lui donne. C'est une sorte d'arrondi. On arrondit en fait toujours au nombre entier supérieur.

floor

C'est l'inverse de la fonction précédente : cette fois, elle renvoie le nombre directement en dessous.

Si vous lui donnez 37.91, la fonction `floor` vous renverra donc 37.

pow

Cette fonction permet de calculer la puissance d'un nombre. Vous devez lui indiquer deux valeurs : le nombre et la puissance à laquelle vous voulez l'élever.

sqrt

Cette fonction calcule la racine carrée d'un nombre. Elle renvoie un `double`.

sin, cos, tan

Ce sont les trois fameuses fonctions utilisées en trigonométrie.

Le fonctionnement est le même, ces fonctions renvoient un `double`.

Ces fonctions attendent une valeur en **radians**.

asin, acos, atan

Ce sont les fonctions arc sinus, arc cosinus et arc tangente, d'autres fonctions de trigonométrie.

Elles s'utilisent de la même manière et renvoient un `double`.

exp

Cette fonction calcule l'exponentielle d'un nombre. Elle renvoie un `double`

log

Cette fonction calcule le logarithme népérien d'un nombre (que l'on note aussi « $\ln$ »).

log10

Cette fonction calcule le logarithme base 10 d'un nombre.